



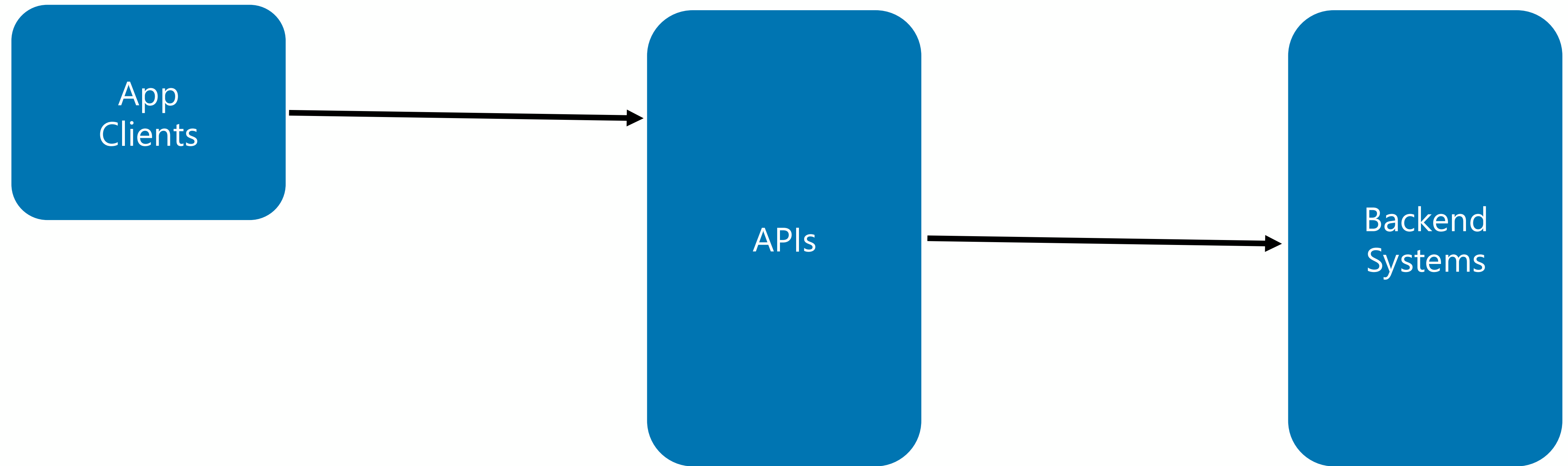
TRAILHEAD
TECHNOLOGY PARTNERS

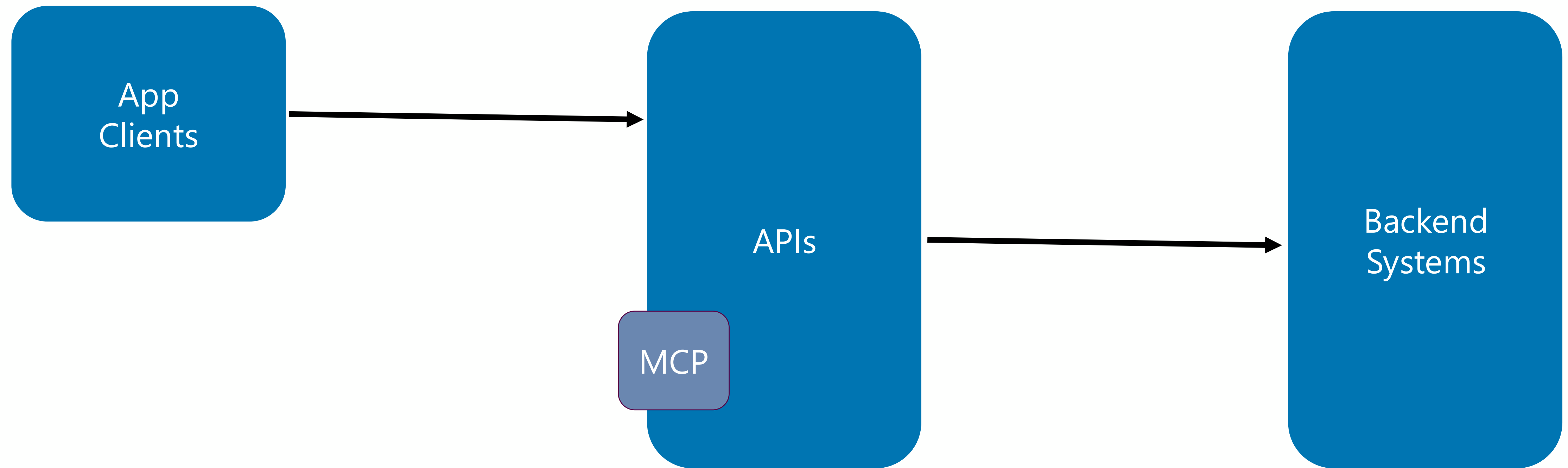
Old API, New Tricks

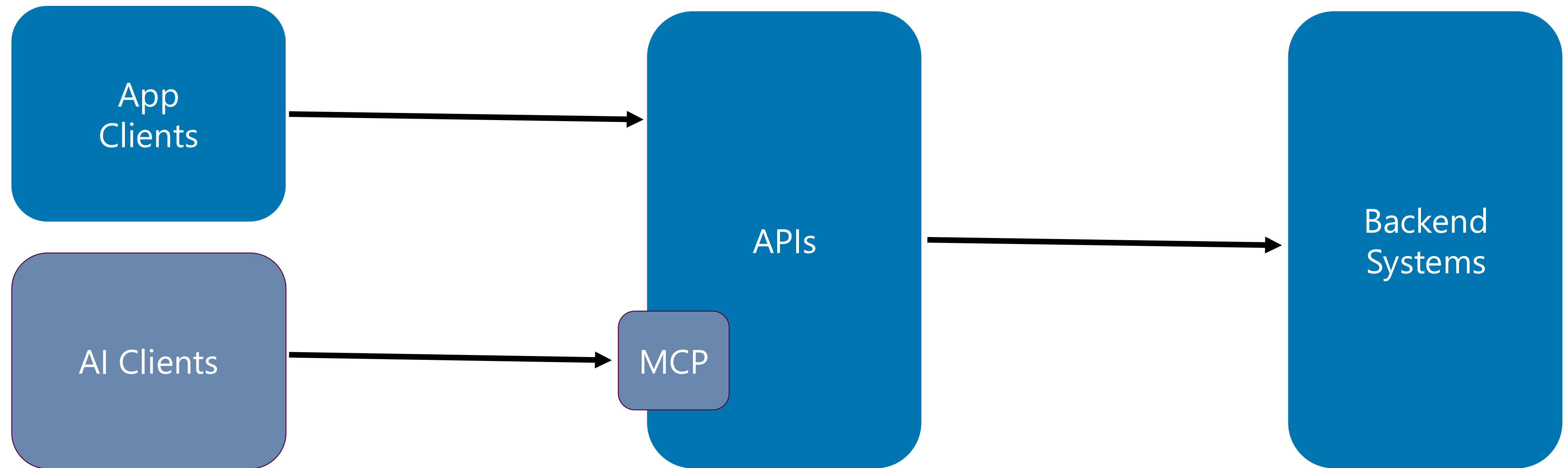
Add MCP to Existing .NET
REST Endpoints



Jonathan "J." Tower







MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

Document generation, assembling contracts from templates and data

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

Document generation, assembling contracts from templates and data

Procurement flow, creating POs, checking budgets, routing approvals

MCP Usage Scenarios

Sales assistant updating CRM, scheduling follow-ups, crafting briefs

Chatbots running reports, aggregating KPIs, summarizing instantly

Customer support copilot, looking up orders, issuing refunds, creating tickets

Document generation, assembling contracts from templates and data

Procurement flow, creating POs, checking budgets, routing approvals

Scheduling assistant, coordinating calendars, proposing times, sending invites

Preview

- 01** What is MCP?
- 02** MCP vs REST
- 03** MCP C# SDK
- 04** Demo + Tool Design
- 05** Debugging & Security

Preview

- 01** What is MCP?
- 02 MCP vs REST
- 03 MCP C# SDK
- 04 Demo + Tool Design
- 05 Debugging & Security

Preview

- 01** What is MCP?
- 02** MCP vs REST
- 03 MCP C# SDK
- 04 Demo + Tool Design
- 05 Debugging & Security

Preview

- 01** What is MCP?
- 02** MCP vs REST
- 03** MCP C# SDK
- 04 Demo + Tool Design
- 05 Debugging & Security

Preview

- 01** What is MCP?
- 02** MCP vs REST
- 03** MCP C# SDK
- 04** Demo + Tool Design
- 05** Debugging & Security

Preview

- 01** What is MCP?
- 02** MCP vs REST
- 03** MCP C# SDK
- 04** Demo + Tool Design
- 05** Debugging & Security

Jonathan "J." Tower



Founding Partner

- 13x Microsoft MVP in .NET and AI
- VP of .NET Foundation
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- [jtowermi](#)
- Jonathan "J." Tower

LET'S TALK



tinyurl.com/th-offer

github.com/trailheadtechnology/dotnet-mcp-rest

Introduction to MCP

“MCP (Model Context Protocol) is an open-source standard introduced by Anthropic for connecting AI to external systems.

“MCP (**Model Context Protocol**) is an open-source standard introduced by Anthropic for connecting AI to external systems.

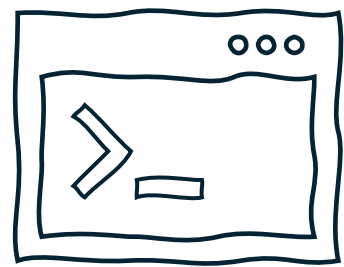
“MCP (**Model Context Protocol**) is an **open-source standard** introduced by Anthropic for connecting AI to external systems.

“MCP (**Model Context Protocol**) is an **open-source standard** introduced by **Anthropic** for connecting AI to external systems.

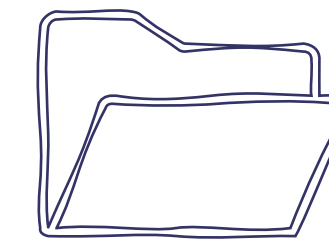
“MCP (Model Context Protocol) is an open-source standard introduced by Anthropic for connecting AI to external systems.

“ **MCP (Model Context Protocol)**
is an **open-source standard**
introduced by **Anthropic**
for **connecting AI to**
external systems.

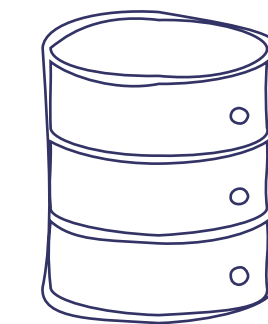
MCP (Model Context Protocol)



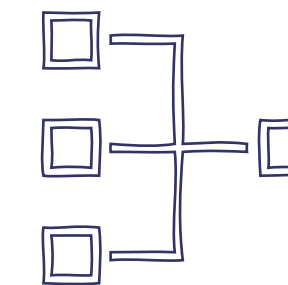
LLM



File System

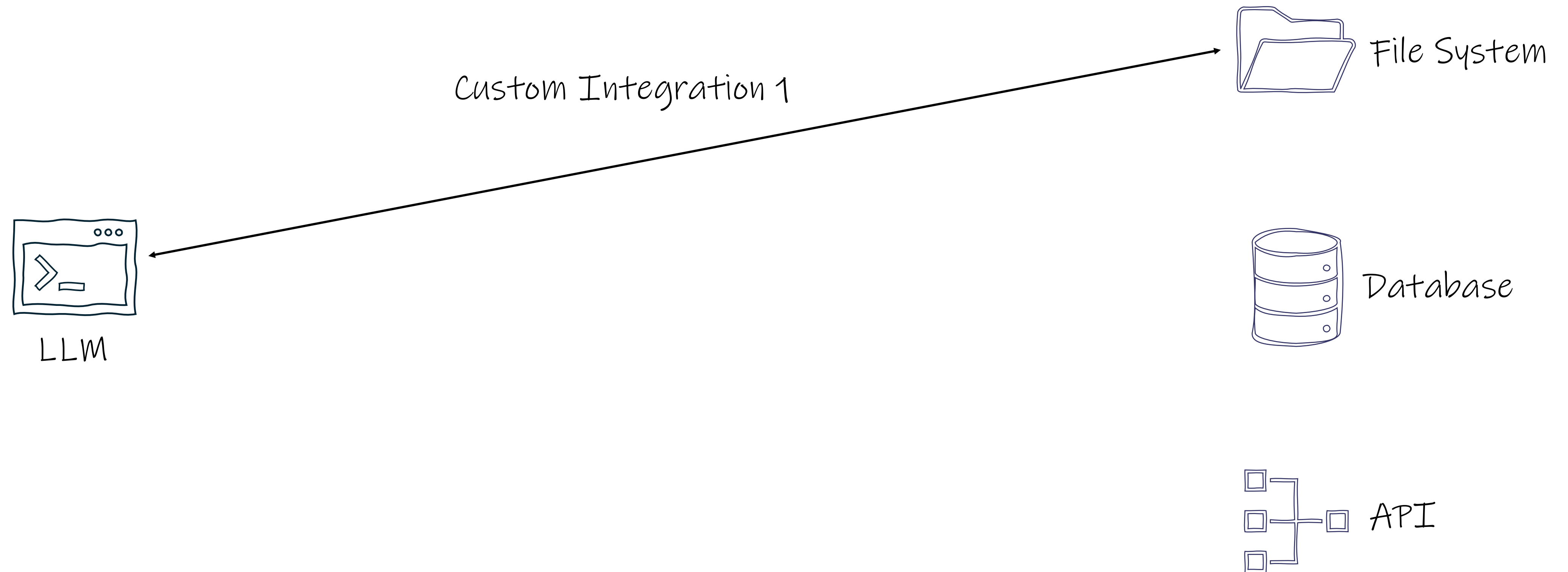


Database

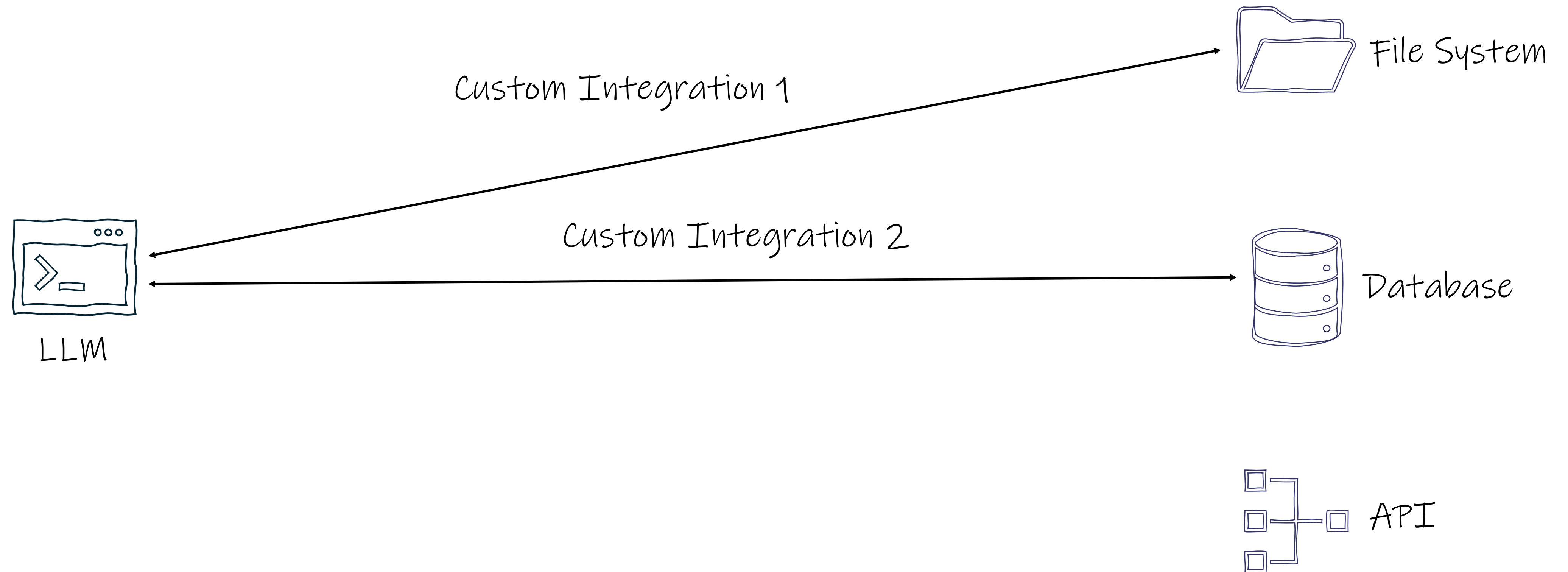


API

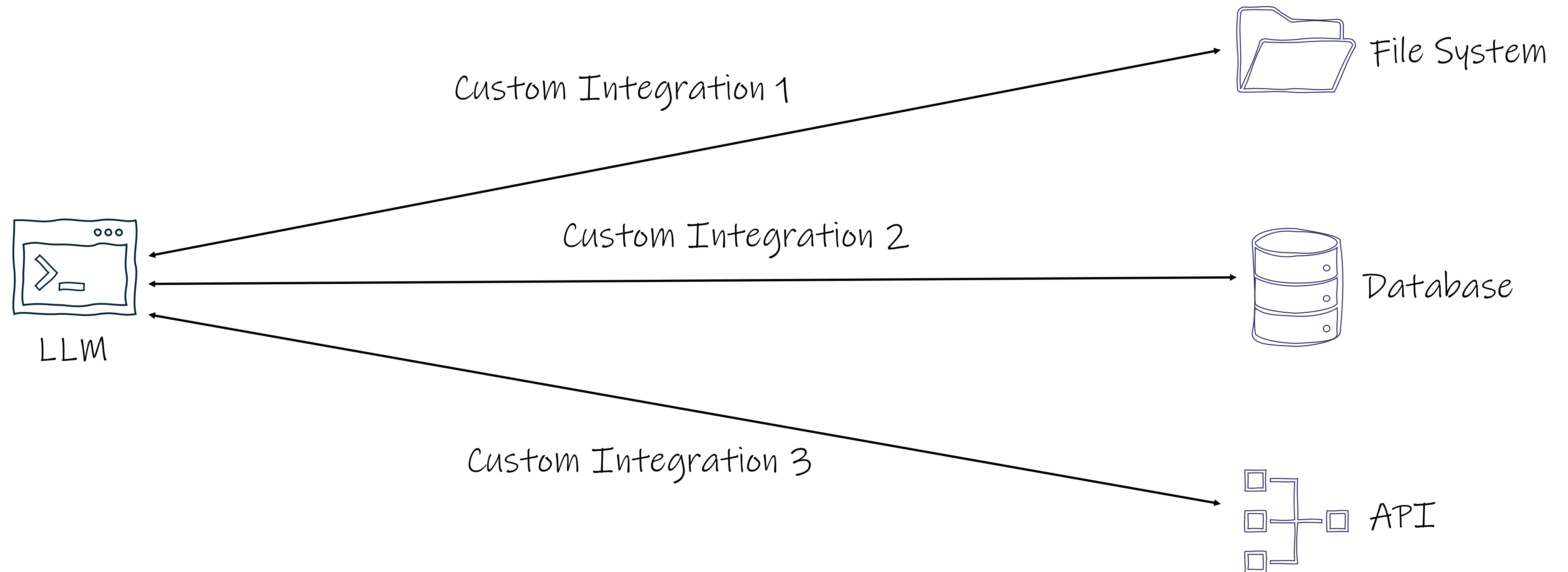
MCP (Model Context Protocol)



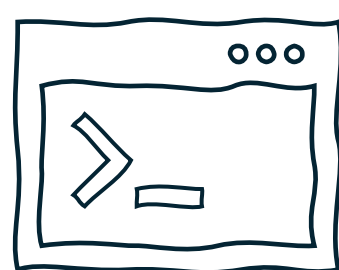
MCP (Model Context Protocol)



MCP (Model Context Protocol)

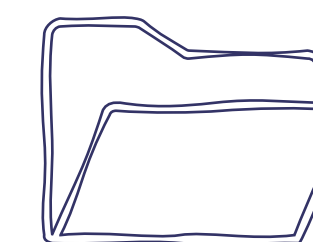


MCP (Model Context Protocol)

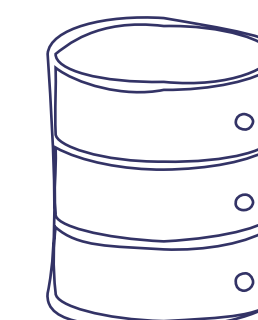


LLM

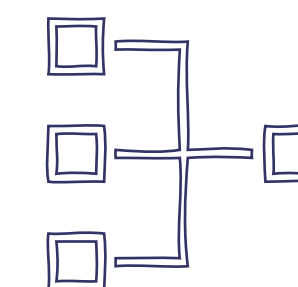
MCP Protocol



File System

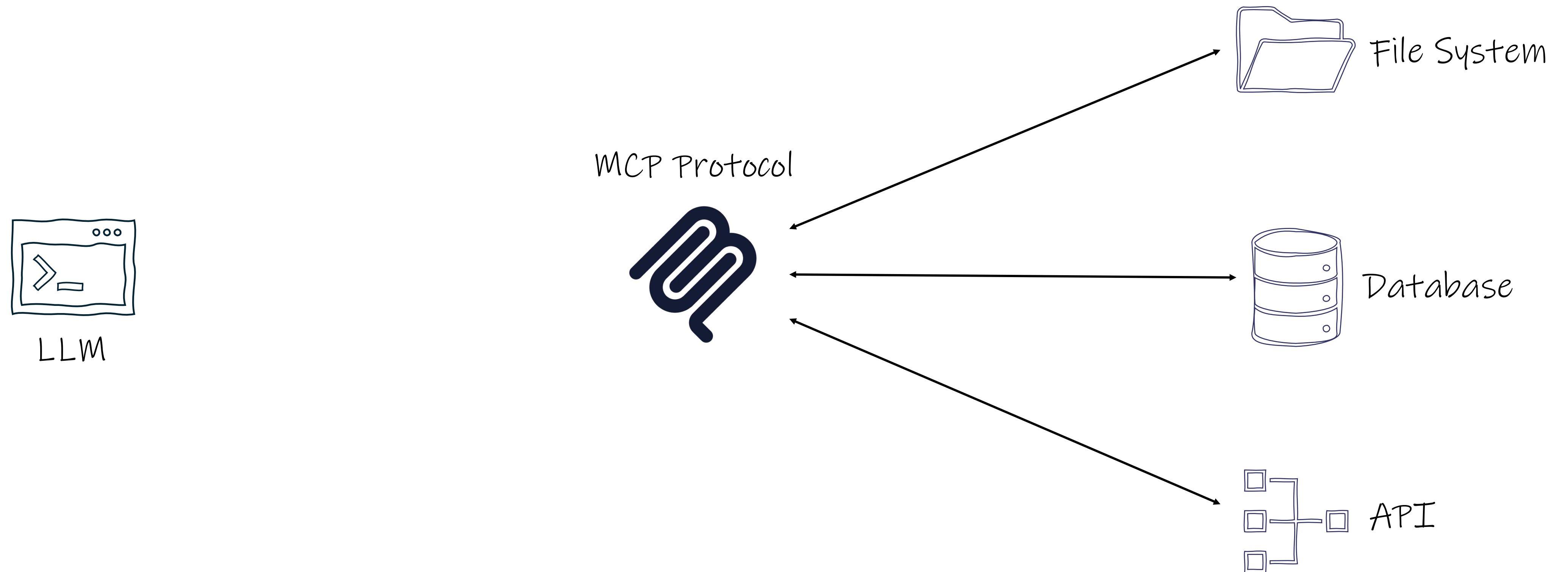


Database

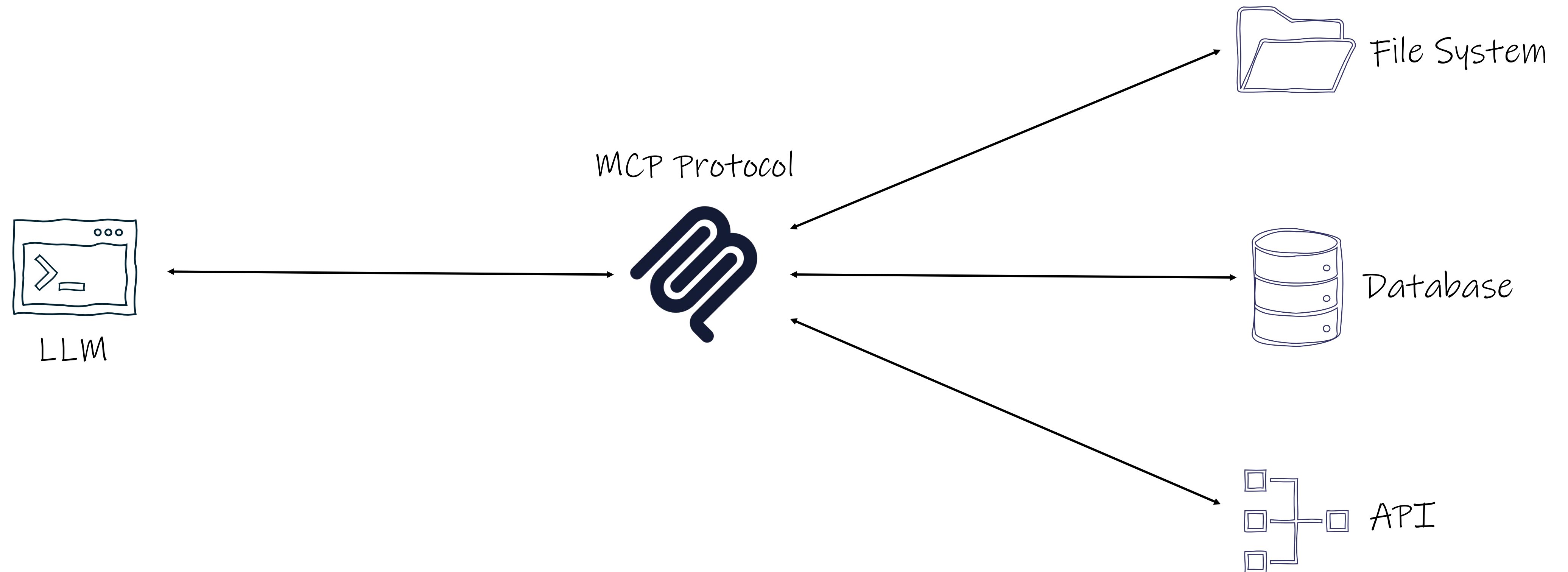


API

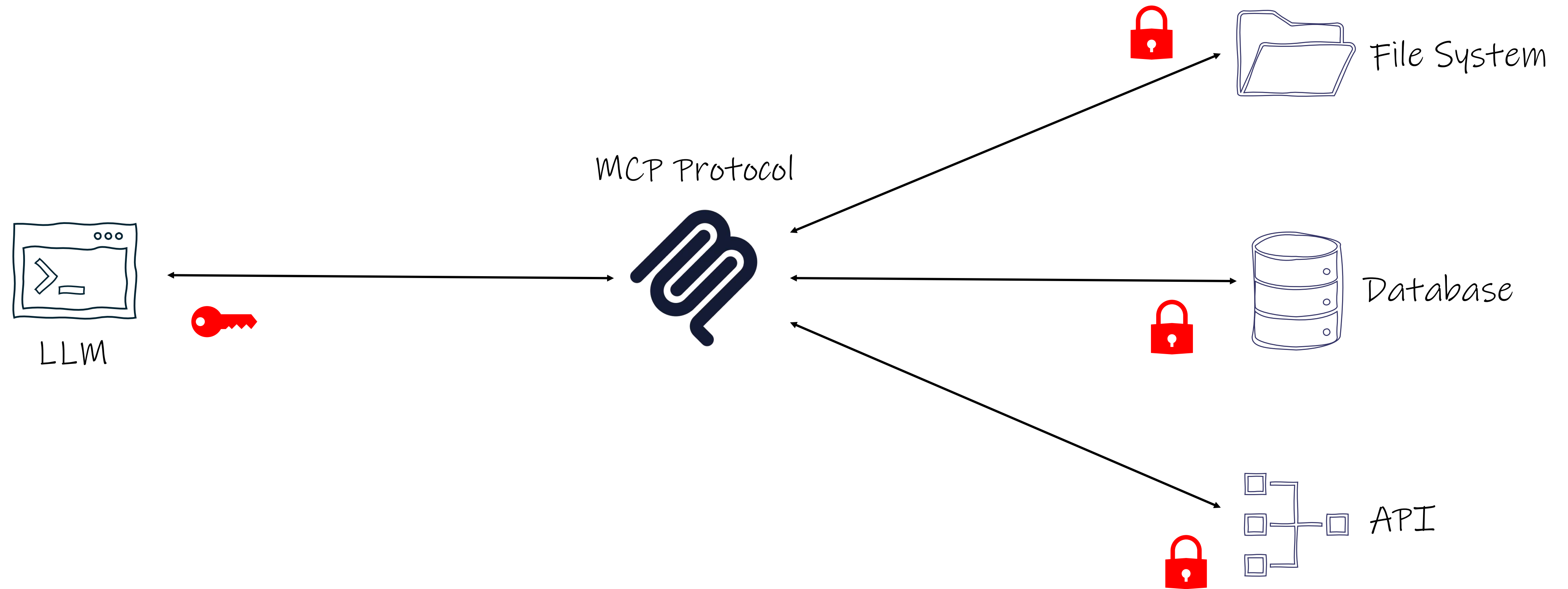
MCP (Model Context Protocol)



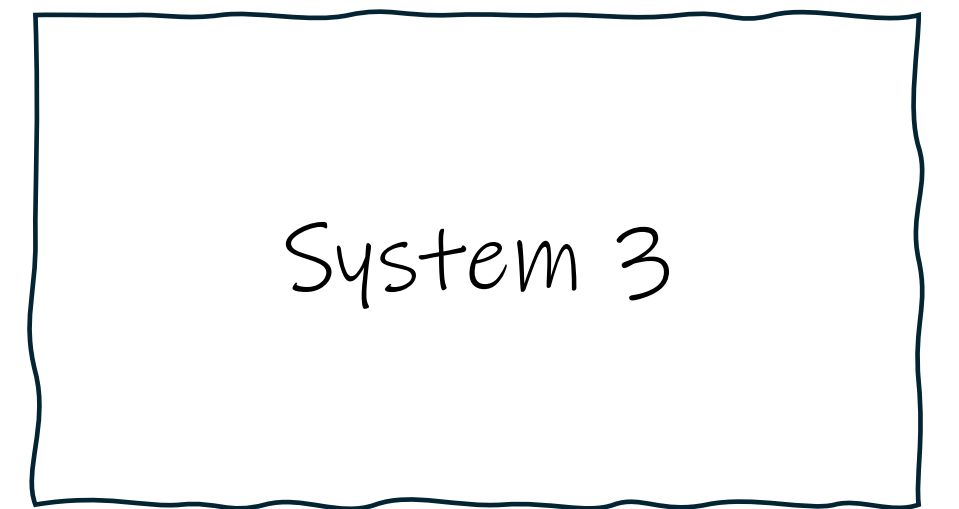
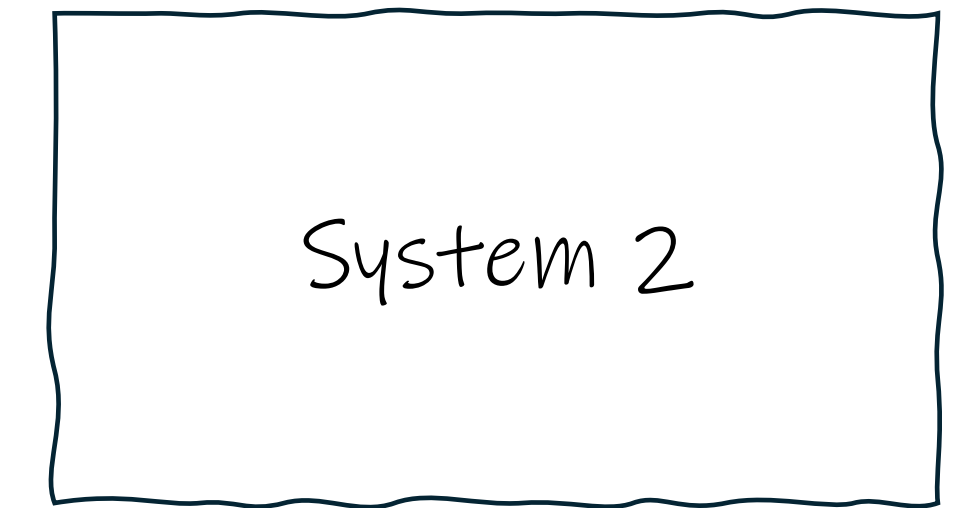
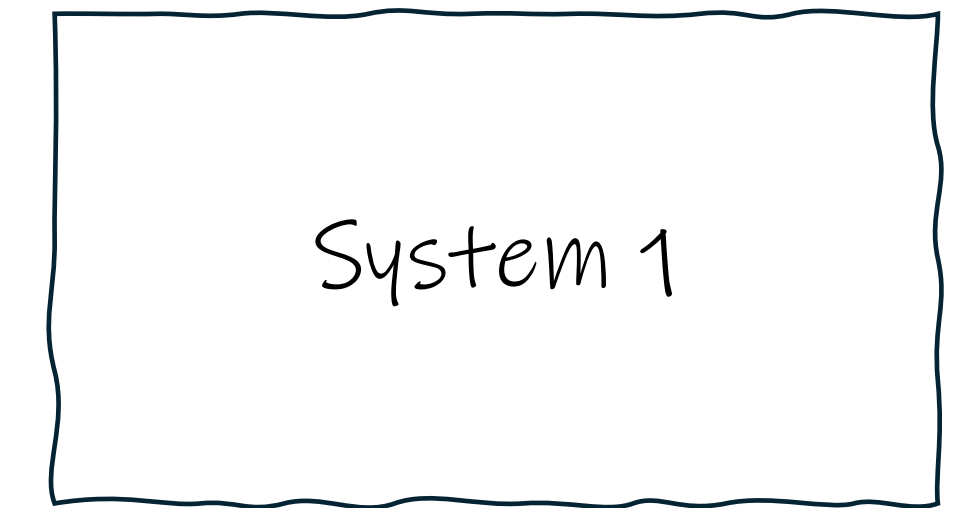
MCP (Model Context Protocol)



MCP (Model Context Protocol)

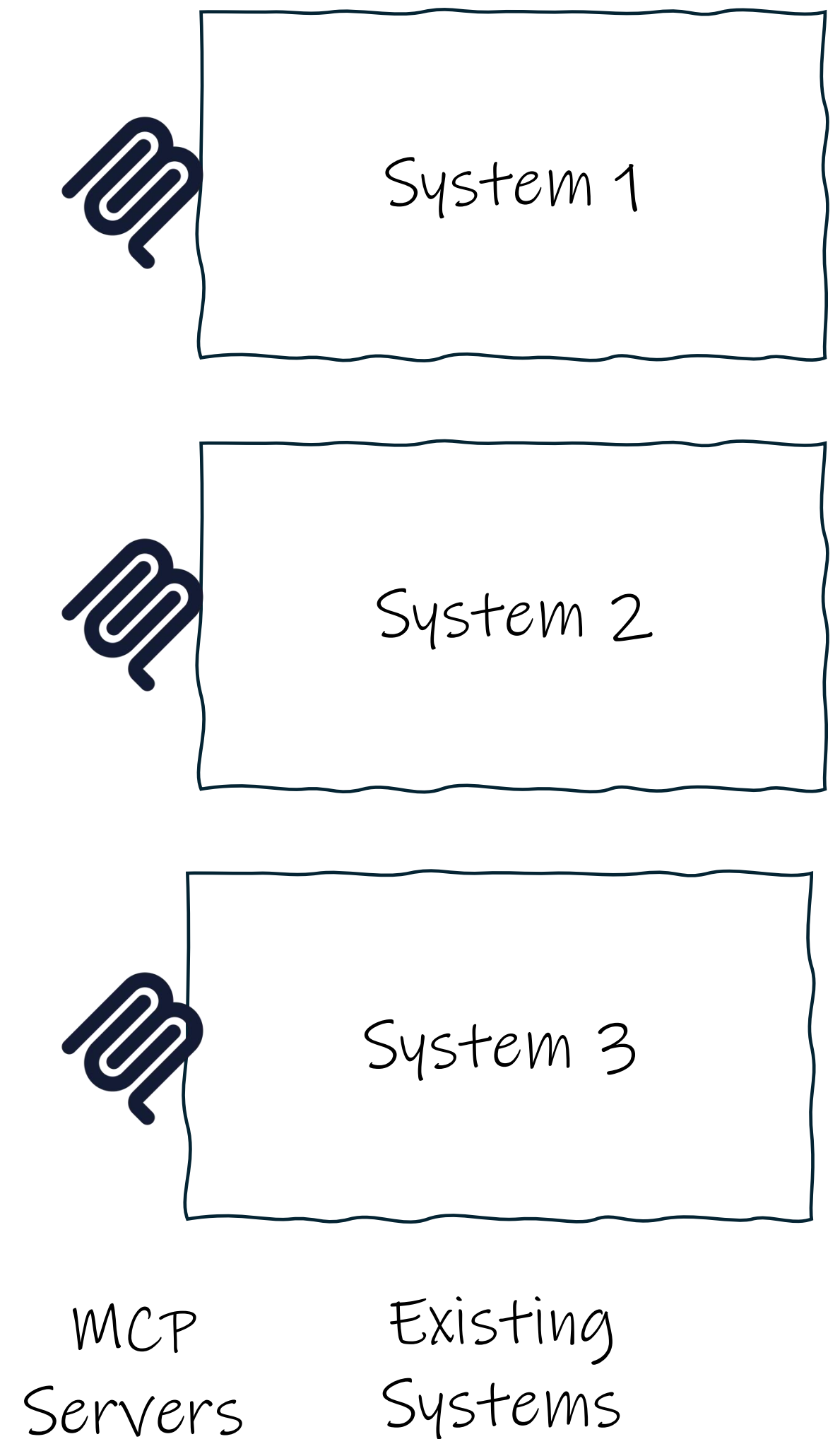


MCP (Model Context Protocol)

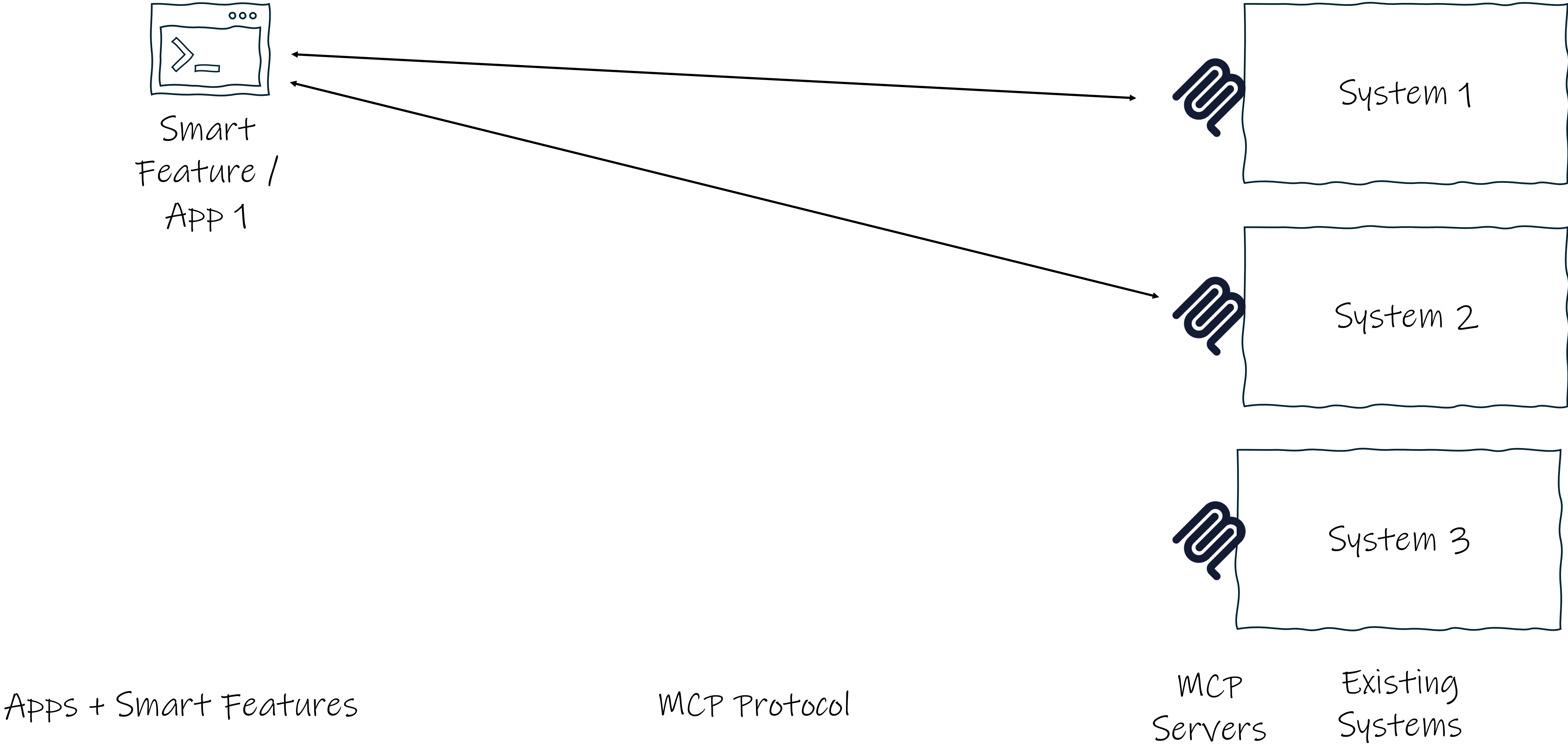


Existing
Systems

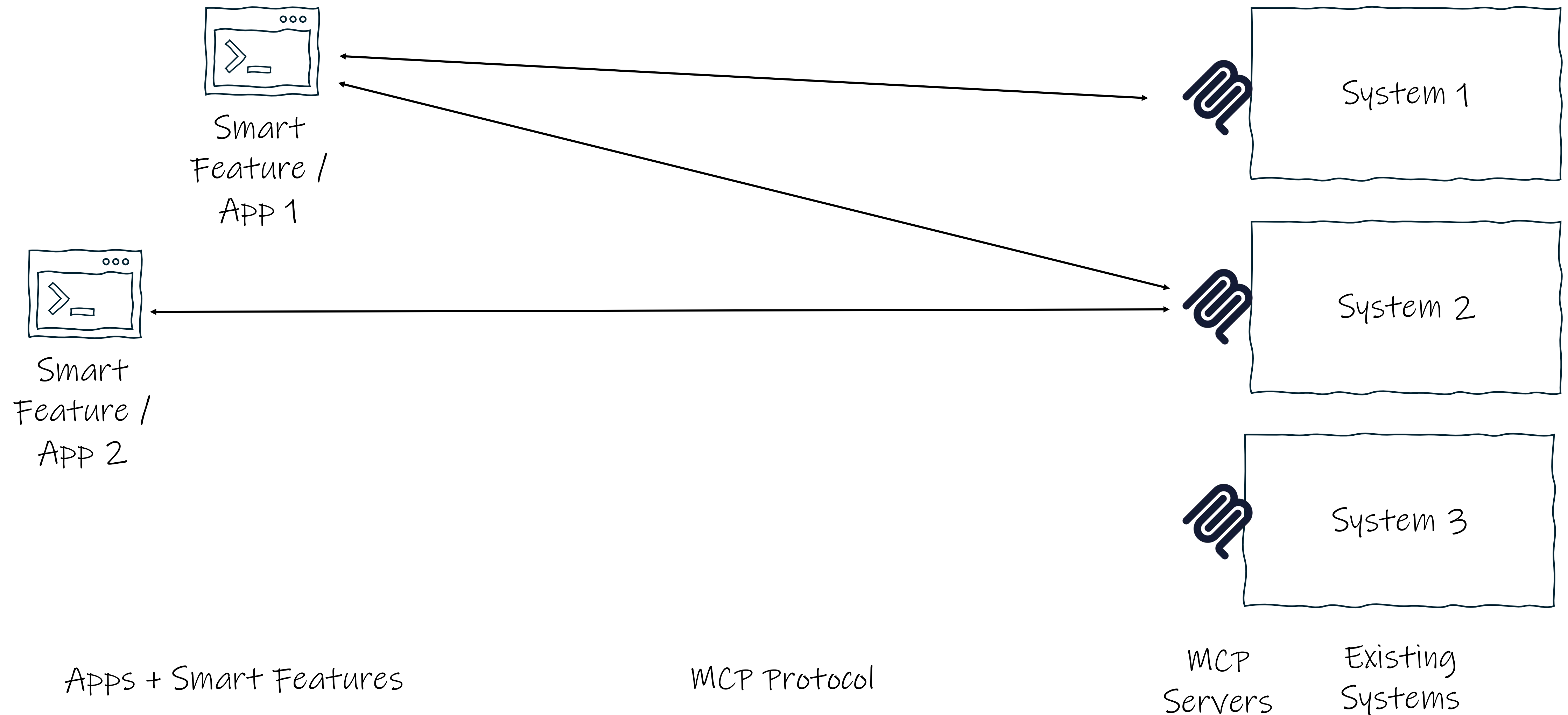
MCP (Model Context Protocol)



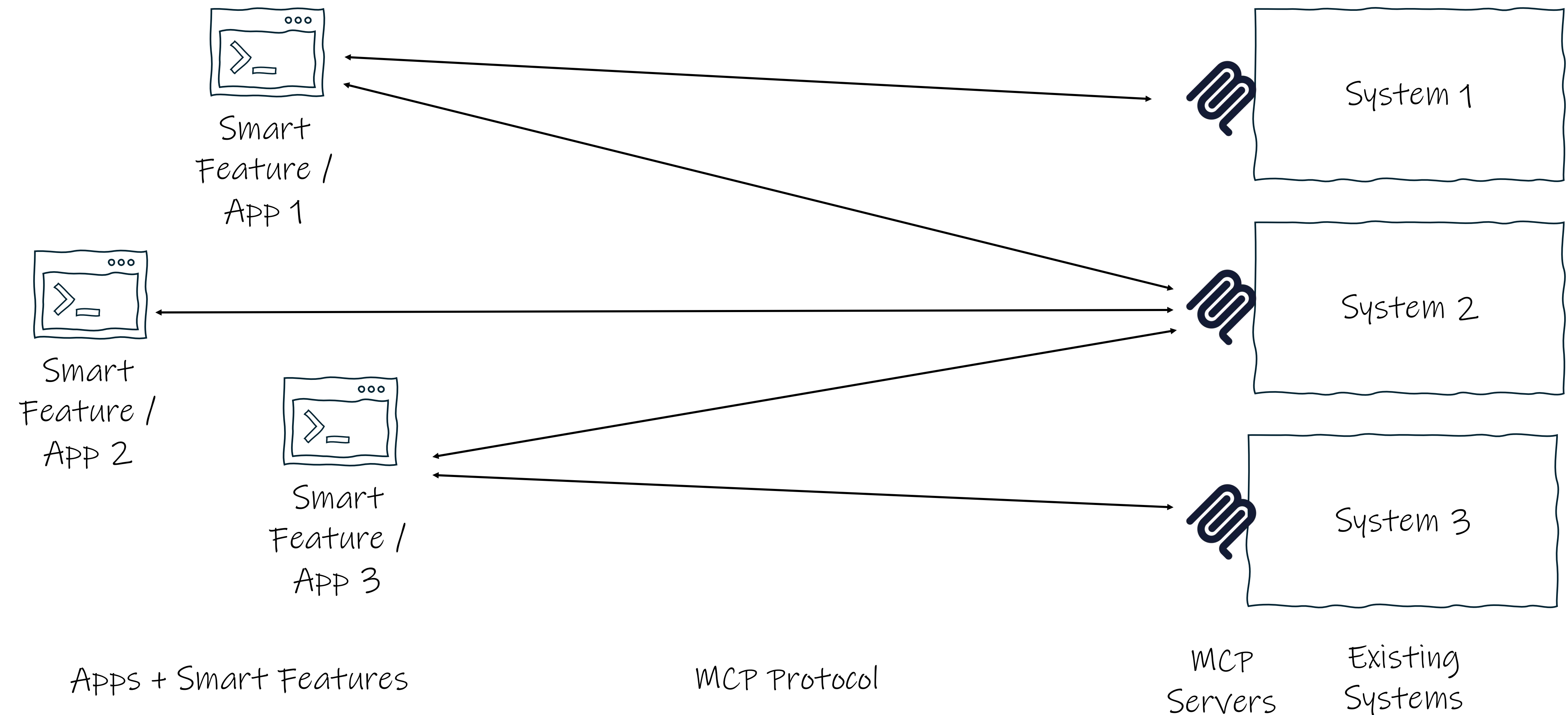
MCP (Model Context Protocol)



MCP (Model Context Protocol)



MCP (Model Context Protocol)



MCP Transports

stdio

Local process over stdin/stout; no network, fast and simple.

Streamable HTTP

HTTP POST in, SSE stream out; remote-friendly, works with auth and proxies (formerly SSE)

Custom

Spec notes custom transports but not standardized

MCP Transports

stdio

Local process over stdin/out; no network, fast and simple.

Streamable HTTP

HTTP POST in, SSE stream out; remote-friendly, works with auth and proxies (formerly SSE)

Custom

Spec notes custom transports but not standardized

MCP Transports

stdio

Local process
over stdin/out;
no network, fast
and simple.

Streamable HTTP

HTTP POST in, SSE stream
out; remote-friendly,
works with auth and
proxies (formerly SSE)

Custom

Spec notes
custom
transports but
not standardized

REST vs MCP

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE integration APIs	Quick AI integration over existing endpoints

REST vs MCP

	REST	MCP
Primary audience	Other developers & services	LLMs/agents (plus humans)
Discovery	OpenAPI/docs (manual; static)	Runtime discovery (list_tools/resources/prompts)
Contract	URIs + HTTP verbs + status codes	Named tools + JSON Schemas + structured results
Invocation & transport	Request/response over HTTP	Per-call tool exec via stdio or streamable HTTP
Orchestration	Client stitches multiple calls	Agent plans multi-step tool chains
Streaming	Ad hoc/rare	First-class partial/progressive outputs
Auth model	HTTP auth per request	Transport-level; server injects downstream auth
Best use	BE/Integration APIs	AI integration over existing endpoints

MCP C# SDK

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

MCP C# SDK Features

01

MCP clients and servers

Build clients and servers in minutes using the SDK's ready-made client and server primitives.

02

Attribute-based discovery

Attribute-based auto-discovery exposes your methods as MCP tools and prompts with a simple scan.

03

MEAI integration

Microsoft.Extensions.AI native. Plug into DI, hosting, and logging you already use in .NET.

04

Transports included

stdio for local; HTTP with SSE for streaming, production-friendly hosting.

05

Bridges to chat stacks

Chat/agent bridges. Surface tools as callable functions in chat client stacks.

06

Works with any MCP server

Server-agnostic clients. Connect to any conformant MCP server without lock-in.

What's New?

01

Spec 2026-07-28 Alignment

v2.0

Realigned to the latest spec: discovery-first negotiation with automatic fallback to the legacy initialize handshake.

04

Resource Subscriptions

v2.1

A subscriptions/listen handler lets clients receive live updates when server resources change.

02

Stateless-by-Default HTTP

v2.0

HTTP serving is now stateless by default — simpler scale-out and load balancing for production hosting.

05

Smarter Transport Fallback

v2.1

Improved HTTP transport negotiation and fallback so clients connect reliably across protocol revisions.

03

Hybrid Session Mode

v2.2

New `HttpServerSessionMode` lets one endpoint serve both older stateful clients and newer stateless clients during migration.

06

Identity Assertion Auth Grant

v1.4

Support for the Identity Assertion Authorization Grant plus security hardening for enterprise auth flows.

What's New?

01

Spec 2026-07-28 Alignment

v2.0

Realigned to the latest spec: discovery-first negotiation with automatic fallback to the legacy initialize handshake.

02

Stateless-by-Default HTTP

v2.0

HTTP serving is now stateless by default — simpler scale-out and load balancing for production hosting.

03

Hybrid Session Mode

v2.2

New `HttpServerSessionMode` lets one endpoint serve both older stateful clients and newer stateless clients during migration.

04

Resource Subscriptions

v2.1

A `subscriptions/listen` handler lets clients receive live updates when server resources change.

05

Smarter Transport Fallback

v2.1

Improved HTTP transport negotiation and fallback so clients connect reliably across protocol revisions.

06

Identity Assertion Auth Grant

v1.4

Support for the Identity Assertion Authorization Grant plus security hardening for enterprise auth flows.

What's New?

01

Spec 2026-07-28 Alignment

v2.0

Realigned to the latest spec: discovery-first negotiation with automatic fallback to the legacy initialize handshake.

02

Stateless-by-Default HTTP

v2.0

HTTP serving is now stateless by default — simpler scale-out and load balancing for production hosting.

03

Hybrid Session Mode

v2.2

New `HttpServerSessionMode` lets one endpoint serve both older stateful clients and newer stateless clients during migration.

04

Resource Subscriptions

v2.1

A `subscriptions/listen` handler lets clients receive live updates when server resources change.

05

Smarter Transport Fallback

v2.1

Improved HTTP transport negotiation and fallback so clients connect reliably across protocol revisions.

06

Identity Assertion Auth Grant

v1.4

Support for the Identity Assertion Authorization Grant plus security hardening for enterprise auth flows.

What's New?

01

Spec 2026-07-28 Alignment

v2.0

Realigned to the latest spec: discovery-first negotiation with automatic fallback to the legacy initialize handshake.

04

Resource Subscriptions

v2.1

A subscriptions/listen handler lets clients receive live updates when server resources change.

02

Stateless-by-Default HTTP

v2.0

HTTP serving is now stateless by default — simpler scale-out and load balancing for production hosting.

05

Smarter Transport Fallback

v2.1

Improved HTTP transport negotiation and fallback so clients connect reliably across protocol revisions.

03

Hybrid Session Mode

v2.2

New `HttpServerSessionMode` lets one endpoint serve both older stateful clients and newer stateless clients during migration.

06

Identity Assertion Auth Grant

v1.4

Support for the Identity Assertion Authorization Grant plus security hardening for enterprise auth flows.

What's New?

01

Spec 2026-07-28 Alignment

v2.0

Realigned to the latest spec: discovery-first negotiation with automatic fallback to the legacy initialize handshake.

04

Resource Subscriptions

v2.1

A subscriptions/listen handler lets clients receive live updates when server resources change.

02

Stateless-by-Default HTTP

v2.0

HTTP serving is now stateless by default — simpler scale-out and load balancing for production hosting.

05

Smarter Transport Fallback

v2.1

Improved HTTP transport negotiation and fallback so clients connect reliably across protocol revisions.

03

Hybrid Session Mode

v2.2

New `HttpServerSessionMode` lets one endpoint serve both older stateful clients and newer stateless clients during migration.

06

Identity Assertion Auth Grant

v1.4

Support for the Identity Assertion Authorization Grant plus security hardening for enterprise auth flows.

What's New?

01

Spec 2026-07-28 Alignment

v2.0

Realigned to the latest spec: discovery-first negotiation with automatic fallback to the legacy initialize handshake.

04

Resource Subscriptions

v2.1

A subscriptions/listen handler lets clients receive live updates when server resources change.

02

Stateless-by-Default HTTP

v2.0

HTTP serving is now stateless by default — simpler scale-out and load balancing for production hosting.

05

Smarter Transport Fallback

v2.1

Improved HTTP transport negotiation and fallback so clients connect reliably across protocol revisions.

03

Hybrid Session Mode

v2.2

New `HttpServerSessionMode` lets one endpoint serve both older stateful clients and newer stateless clients during migration.

06

Identity Assertion Auth Grant

v1.4

Support for the Identity Assertion Authorization Grant plus security hardening for enterprise auth flows.

DEMO

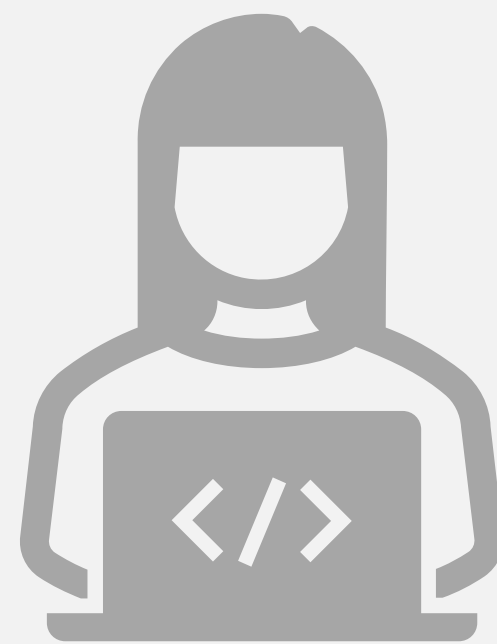
Tool Design Quality

Descriptions Are Your API Contract

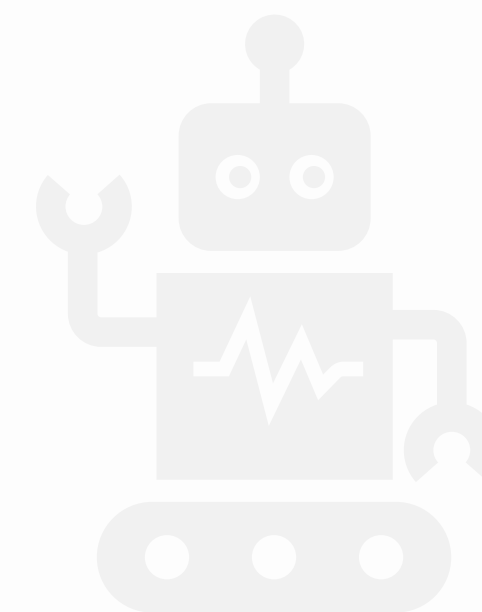
GET /api/todos

POST /api/todos

PATCH /api/todos/{id}/complete

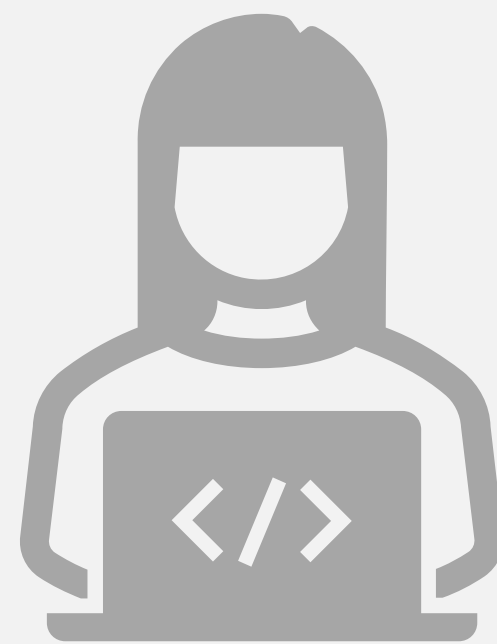


```
[Description("Returns all todos. Pass  
true for completed, false for  
incomplete, or omit to get all.")]
```

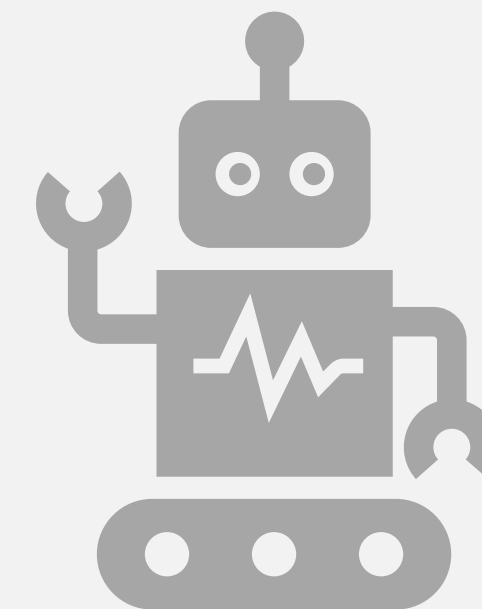


Descriptions Are Your API Contract

```
GET /api/todos  
POST /api/todos  
PATCH /api/todos/{id}/complete
```



```
[Description("Returns all todos. Pass  
true for completed, false for  
incomplete, or omit to get all.")]
```



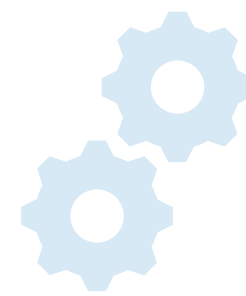
Anatomy of a Good Description



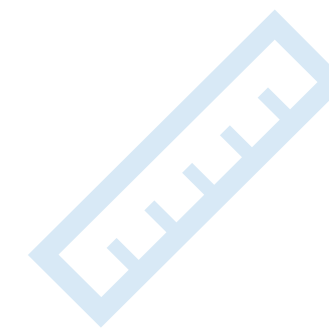
What it does



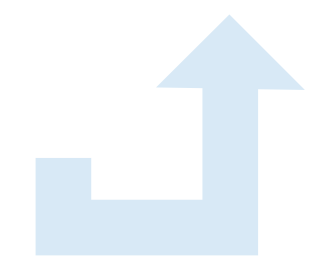
When to use it



Valid param values



Param units and
format



Return value
expectation

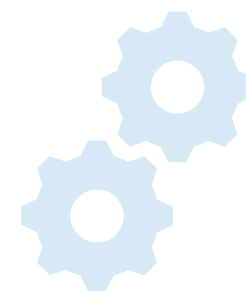
Anatomy of a Good Description



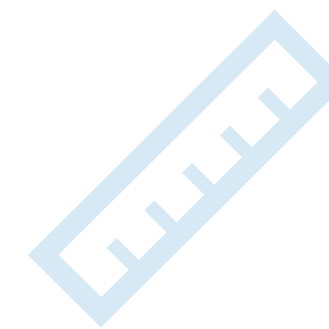
What it does



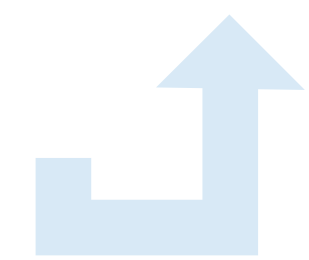
When to use it



Valid param values



Param units and
format



Return value
expectation

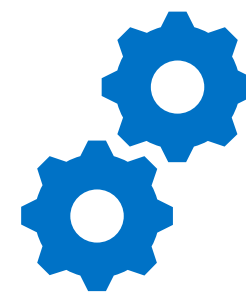
Anatomy of a Good Description



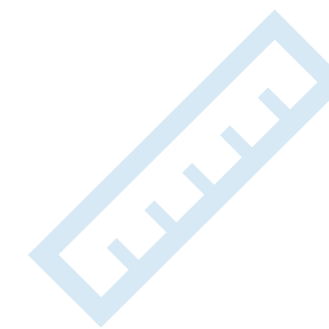
What it does



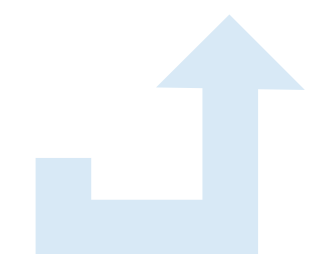
When to use it



Valid param values



Param units and
format



Return value
expectation

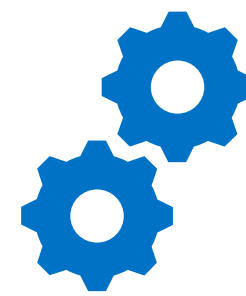
Anatomy of a Good Description



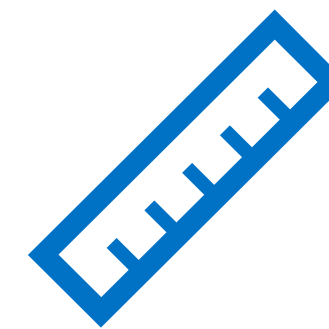
What it does



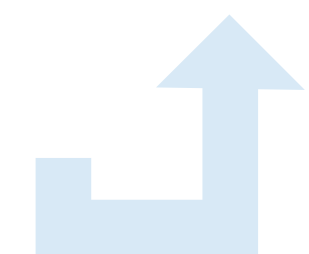
When to use it



Valid param values



Param units and
format



Return value
expectation

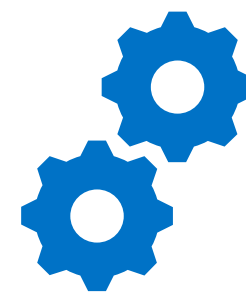
Anatomy of a Good Description



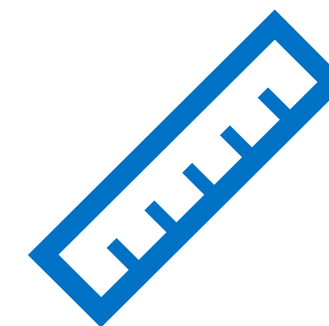
What it does



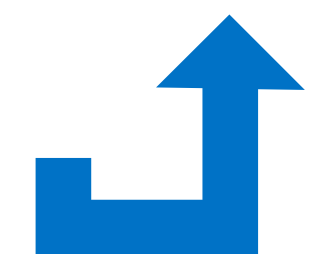
When to use it



Valid param values



Param units and
format



Return value
expectation

Bad Description ✖

```
[Description("Gets todos")]  
public string GetTodos(bool? complete)
```

Bad Description ✖

```
[Description("Gets todos")]  
public string GetTodos(bool? complete)
```

Too simple

Bad Description ✖

```
[Description("Gets todos")]  
public string GetTodos(bool? complete)
```

Too simple

Missing

Bad Description ❌

```
[Description("This tool is used to retrieve, fetch, get, list, or otherwise " +  
    "obtain the todo items (also known as tasks, to-dos, action items, or " +  
    "reminders) that are currently stored in the TodoStore, which is an " +  
    "in-memory static store backed by a ConcurrentDictionary<int, Todo> that " +  
    "was introduced in version 1.0 of the TodoAPI project when we migrated " +  
    "away from the original List<Todo> implementation for thread-safety " +  
    "reasons. It returns the items as a JSON-serialized array of Todo " +  
    "objects, where each Todo object contains an Id (integer, auto-" +  
    "incremented starting at 1), a Title (string, the text of the todo), " +  
    "and an IsComplete flag (boolean). Note that this tool does NOT add" +  
    "update, delete, complete, or uncomplete todos; for those operations " +  
    ...
```

Bad Description ❌

Too long

```
[Description("This tool is used to retrieve, fetch, get, list, or otherwise " +  
  "obtain the todo items (also known as tasks, to-dos, action items, or " +  
  "reminders) that are currently stored in the TodoStore, which is an " +  
  "in-memory static store backed by a ConcurrentDictionary<int, Todo> that " +  
  "was introduced in version 1.0 of the TodoAPI project when we migrated " +  
  "away from the original List<Todo> implementation for thread-safety " +  
  "reasons. It returns the items as a JSON-serialized array of Todo " +  
  "objects, where each Todo object contains an Id (integer, auto-" +  
  "incremented starting at 1), a Title (string, the text of the todo), " +  
  "and an IsComplete flag (boolean). Note that this tool does NOT add" +  
  "update, delete, complete, or uncomplete todos; for those operations " +  
  ...
```

Good Description

```
[Description("Returns all todo items. Pass true to get only  
completed items, false for only incomplete items, or omit  
the parameter entirely to return all items regardless of  
status.")]
```

```
public string GetTodos(  
    [Description("Filter by completed status. true = completed  
only, false = incomplete only, omit = all items")]  
    bool? complete = null)
```

Good Description



What it does;
What it returns

```
[Description("Returns all todo items. Pass true to get only  
completed items, false for only incomplete items, or omit  
the parameter entirely to return all items regardless of  
status.")]
```

```
public string GetTodos(  
    [Description("Filter by completed status. true = completed  
only, false = incomplete only, omit = all items")]  
    bool? complete = null)
```

Good Description



What it does;
What it returns

What params do

```
[Description("Returns all todo items. Pass true to get only  
completed items, false for only incomplete items, or omit  
the parameter entirely to return all items regardless of  
status.")]
```

```
public string GetTodos(  
    [Description("Filter by completed status. true = completed  
only, false = incomplete only, omit = all items")]  
    bool? complete = null)
```

Good Description



What it does;
What it returns

What params do

```
[Description("Returns all todo items. Pass true to get only  
completed items, false for only incomplete items, or omit  
the parameter entirely to return all items regardless of  
status.")]
```

Example param values

```
public string GetTodos(  
    [Description("Filter by completed status. true = completed  
only, false = incomplete only, omit = all items")]  
    bool? complete = null)
```

Debugging MCP

MCP Inspector

- ✗ curl
- ✗ OpenAPI / Swagger
- ✗ Postman

MCP Inspector

- ✗ curl
- ✗ OpenAPI / Swagger
- ✗ Postman

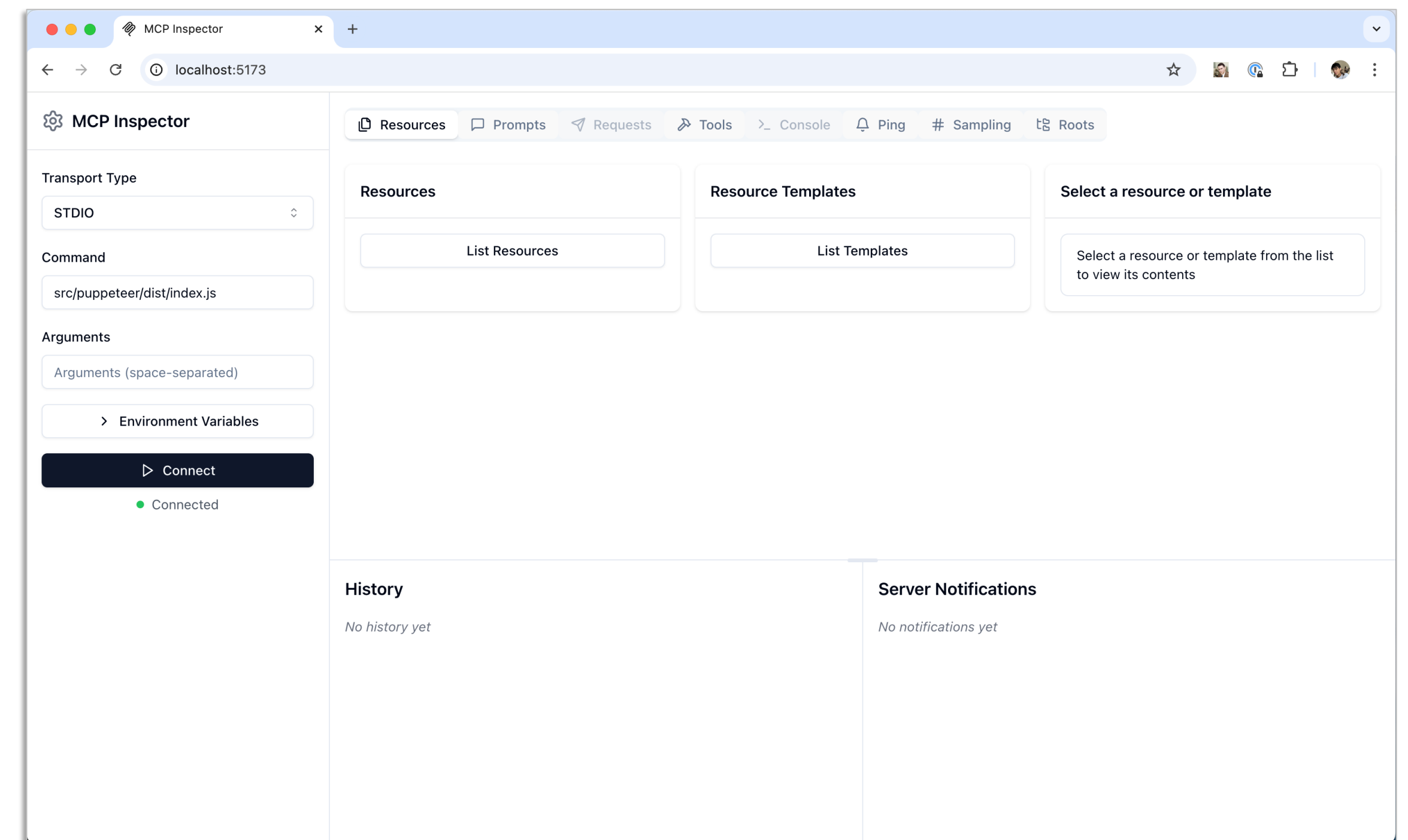
```
> npx @modelcontextprotocol/inspector <command>
```



MCP Inspector

- ✗ curl
- ✗ OpenAPI / Swagger
- ✗ Postman

```
> npx @modelcontextprotocol/inspector <command>
```



DEMO

Security

From "Trust Me" to Enterprise-Grade



TRAILHEAD
TECHNOLOGY PARTNERS

Early MCP “Security”

- Every server **rolled its own**
- API keys in config files; stdio and localhost as the “security model”
- Fine for demos and local dev, but a non-starter for enterprise rollouts



Early MCP “Security”

- Every server **rolled its own**
- API keys in config files; **stdio** and localhost as the “security model”
- Fine for demos and local dev, but a non-starter for enterprise rollouts



Early MCP “Security”

- Every server **rolled its own**
- API keys in config files; **stdio** and localhost as the “security model”
- Fine for demos and local dev, but a **non-starter for enterprise** rollouts



Now: Enterprise-Grade Security in MCP

- **OAuth 2.1** authorization is part of the MCP spec
- User identity flows through to downstream APIs
- Audience-bound tokens and security hardening baked into the SDK
- Stateless HTTP means gateways, WAF, APIM just work

Now: Enterprise-Grade Security in MCP

- **OAuth 2.1** authorization is part of the MCP spec
- **User identity flows through** to downstream APIs
- Audience-bound tokens and security hardening baked into the SDK
- Stateless HTTP means gateways, WAF, APIM just work

Now: Enterprise-Grade Security in MCP

- **OAuth 2.1** authorization is part of the MCP spec
- **User identity flows through** to downstream APIs
- **Audience-bound** tokens and security hardening baked into the SDK
- Stateless HTTP means gateways, WAF, APIM just work

Now: Enterprise-Grade Security in MCP

- **OAuth 2.1** authorization is part of the MCP spec
- **User identity flows through** to downstream APIs
- **Audience-bound** tokens and security hardening baked into the SDK
- **Stateless HTTP** means gateways, WAF, APIM just work

DEMO

Summing Up

1. MCP **can be bolted on** to existing REST APIs
2. While REST exposes **resources**, MCP exposes **capabilities** (not always 1:1)
3. Audience for REST is **devs**; for MCP it's **AI**s
4. LLMs can do **multi-step orchestration**
5. Tool descriptions are **contracts**, not just **documentation**
6. Debug with **MCP Inspector** + secure with **OAuth**



Summing Up

1. MCP **can be bolted on** to existing REST APIs
2. While REST exposes **resources**, MCP exposes **capabilities** (not always 1:1)
3. Audience for REST is **devs**; for MCP it's **AI**s
4. LLMs can do **multi-step orchestration**
5. Tool descriptions are **contracts**, not just **documentation**
6. Debug with **MCP Inspector** + secure with **OAuth**



Summing Up

1. MCP **can be bolted on** to existing REST APIs
2. While REST exposes **resources**, MCP exposes **capabilities** (not always 1:1)
3. Audience for REST is **devs**; for MCP it's **AI**s
4. LLMs can do **multi-step orchestration**
5. Tool descriptions are **contracts**, not just **documentation**
6. Debug with **MCP Inspector** + secure with **OAuth**



Summing Up

1. MCP **can be bolted on** to existing REST APIs
2. While REST exposes **resources**, MCP exposes **capabilities** (not always 1:1)
3. Audience for REST is **devs**; for MCP it's **AI**s
4. LLMs can do **multi-step orchestration**
5. Tool descriptions are **contracts**, not just **documentation**
6. Debug with **MCP Inspector** + secure with **OAuth**



Summing Up

1. MCP **can be bolted on** to existing REST APIs
2. While REST exposes **resources**, MCP exposes **capabilities** (not always 1:1)
3. Audience for REST is **devs**; for MCP it's **AI**s
4. LLMs can do **multi-step orchestration**
5. Tool descriptions are **contracts**, not just **documentation**
6. Debug with **MCP Inspector** + secure with **OAuth**



Summing Up

1. MCP **can be bolted on** to existing REST APIs
2. While REST exposes **resources**, MCP exposes **capabilities** (not always 1:1)
3. Audience for REST is **devs**; for MCP it's **AI**s
4. LLMs can do **multi-step orchestration**
5. Tool descriptions are **contracts**, not just **documentation**
6. Debug with **MCP Inspector** + secure with **OAuth**



Thanks! Questions?

Jonathan "J." Tower

- 13x Microsoft MVP in .NET and AI
- VP of .NET Foundation
- jtower@trailheadtechnology.com
- trailheadtechnology.com/blog
- [jtowermi](https://twitter.com/jtowermi)
- Jonathan "J." Tower

LET'S TALK



tinyurl.com/th-offer

github.com/trailheadtechnology/dotnet-mcp-rest